

NetApp (2025-2026)

CVS-QA Test Automation Leadership

Leadership Principles Demonstrated:

- **Ownership** - Took responsibility beyond my assigned tasks to ensure team success
- **Invent and Simplify** - Created documentation that simplified a 2-week process to 2 days
- **Hire and Develop the Best** - Invested time in enabling teammates to be productive
- **Insist on the Highest Standards** - Maintained highest testing throughput while helping others
- **Bias for Action** - Didn't wait for someone else to create documentation; took initiative
- **Earn Trust** - Teammates thanked me and continue to rely on the resource
- **Deliver Results** - Exceeded personal goals while enabling team success

Situation

Our team was tasked with creating test automation for CVS-QA, a critical 3-month initiative. Initially only 2 engineers were assigned, eventually scaling to ~6. The team lacked standardized setup procedures, and the environment configuration was complex—taking me nearly 2 weeks to figure out independently.

Task

I needed to not only complete my assigned testing work but also help the team ramp up efficiently and establish best practices for environment setup. Our success depended on the entire team being productive quickly and maintaining high throughput to meet our testing goals.

Action

- Became the first engineer to fully configure a test environment, documenting every challenge and solution along the way
- Provided hands-on setup assistance to teammates, nearly fully configuring environments for Zach and Cole
- Collaborated closely with Nathaniel Hoefer to understand the CVS-QA change process and translated this knowledge for the team (also updating docs)
- Created a comprehensive Confluence guide for test environment setup that became the team's primary reference
- Maintained one of the highest testing throughputs, completing all assigned work and volunteering for additional tasks

Result

- Reduced environment setup time by 85% (from ~2 weeks to ~2 days) for all subsequent team members
 - Completed ~ personally—more than double the team average of 5
 - Created documentation still heavily used today by every team member
 - Enabled the team to meet our testing goal—which would not have been achievable without these contributions
 - Established myself as a technical leader and force multiplier for team productivity
-

Strategic Feature Analysis - SVM Peering Retry Cancellation

Leadership Principles Demonstrated:

- **Customer Obsession** - Recognized customers already had a viable solution; focused on real needs
- **Ownership** - Took responsibility for challenging a feature I was assigned to build
- **Are Right, A Lot** - Made correct judgment call that saved 12-16 weeks of effort
- **Think Big** - Looked beyond immediate task to overall product strategy and timeline
- **Have Backbone; Disagree and Commit** - Challenged the roadmap with data and conviction
- **Dive Deep** - Conducted thorough architectural analysis before making recommendation
- **Frugality** - Prevented wasteful spending of engineering resources on low-ROI work
- **Deliver Results** - Enabled team to focus on features that actually shipped on time

Situation

During the FlexCache AGA 2 development cycle, I was assigned sole ownership of the SVM peering retry feature. The team was under immense pressure to deliver, as several features had already been delayed. This feature required deep changes across CVS architecture—APIs, and ONTAP integration.

Task

My goal was to deliver the SVM peering retry functionality for AGA 2. However, as I dove into the technical requirements, I needed to assess whether this feature was the right investment given our tight timeline and customer needs.

Action

- Conducted deep architectural research across CVS systems, documenting areas requiring changes and job flow patterns
- Created a complete functional spec for product and engineering stakeholders to evaluate scope
- Analyzed ROI and determined this would require 3-4 sprints with minimal customer value, as customers already had a viable retry path
- Proactively scheduled a meeting with senior leadership (Mohammed, Sujeetha, Saravana, Hrishi, and Danny) to present findings and re-evaluate priority
- Facilitated alignment among stakeholders that a dedicated retry function was not needed

Result

- Saved 3-4 sprints of engineering effort (approximately 12-16 weeks of work) by canceling an unnecessary feature
 - Enabled the team to focus on higher-value AGA 2 features that customers were actively requesting
 - Prevented scope creep that would have jeopardized our already-tight deadline
 - Demonstrated strategic thinking beyond just execution—analyzing business value and influencing senior leadership decisions
 - This discovery and resulting cancellation were critical to meeting our AGA 2 deadline
-

AGA 2 FlexCache Delivery - War Room Leadership & Crisis Resolution

Leadership Principles Demonstrated:

- **Customer Obsession** - Focused on delivering ANF-compatible APIs customers were waiting for
- **Ownership** - Stepped up to lead despite not being most senior; owned the outcome
- **Invent and Simplify** - Refactored dual-job architecture into cleaner single-job pattern
- **Are Right, A Lot** - Correctly diagnosed complex state propagation and job orchestration issues
- **Insist on the Highest Standards** - Refused to ship with known defects; drove quality fixes
- **Bias for Action** - Led 12-hour war room and authored all PRs to resolve immediately
- **Earn Trust** - Team relied on me to lead crisis despite being Level 3 among Level 4s
- **Dive Deep** - Debugged across multiple layers: API, executor, database, ONTAP integration
- **Deliver Results** - Resolved three critical defects and shipped on time to waiting customers

Situation

Two weeks before the AGA 2 FlexCache release deadline, three critical defects surfaced that threatened our delivery. This release was essential—many customers were waiting for FlexCache features, and we had already experienced delays on previous features. A 12-hour war room was convened with ~5 engineers to diagnose and resolve the issues. The defects included: (1) NFSAAAS-145645 - Volume update API creating dual jobs incompatible with ANF, (2) Cache volume bytes_used reporting - Incorrect logical-used vs physical-used metrics for cache volumes, and (3) NFSAAAS-145646 - FlexCache prepopulate job state transitions not propagating from ONTAP to CVP API.

Task

As a Level 3 engineer working alongside Level 4 peers, I needed to help identify root causes, implement fixes, and ensure we met our customer commitments. Despite not being the most senior engineer present, the situation required immediate leadership and technical execution to resolve complex API, job orchestration, and state management issues.

Action

- Stepped up to lead the war room, driving conversations and coordinating the debugging effort across the 12-hour session
- Led the triage process to isolate root causes across three distinct but related issues
- Identified the dual-job architecture problem (NFSAAAS-145645): Diagnosed that the volume update API was creating 2 jobs simultaneously (update + prepopulate), returning 2 job IDs incompatible with ANF standards
- Designed the refactored solution: Proposed implementing a chained-job pattern (aligned with deleteBaseSnapshot pattern) where the API creates a single update job, which then asynchronously schedules prepopulate via the executor
- Independently diagnosed the state propagation bug (NFSAAAS-145646): Identified that prepopulate job state transitions (IN_PROGRESS → COMPLETE/ERROR) were not being propagated from ONTAP to CVP API, causing cachePrePopulateState to remain stuck at "IN_PROGRESS" even after ONTAP jobs completed successfully
- Implemented the fix for state management: Created the solution to properly reflect prepopulate job states via cachePrePopulateState and expose via GET /volumes/{id}
- Collaborated on the bytes_used reporting issue: Worked with the team to implement special handling for cache volumes to report physical-used rather than logical-used values (GCNV-only, no ANF impact)
- Authored every PR required for the RCHF and drove them to completion with proper testing
- Coordinated cross-component fixes spanning CVP API, job executor, database, and ONTAP integration

Result

- Successfully resolved all three critical defects within the 12-hour war room session
 - Met the AGA 2 release deadline with ANF-compatible single-job architecture
 - Delivered accurate cache volume metrics and proper prepopulate state management to customers
 - Prevented customer-facing issues by catching state propagation bugs before release
 - Demonstrated leadership beyond my level, taking charge in a crisis situation alongside more senior engineers
 - Ensured release success—without these contributions, we would have missed our deadline for the second release in a row
 - Proved capability to perform under extreme pressure, lead technical crisis resolution, and navigate complex distributed system debugging
-

Acting Team Lead (25/75 - Leadership/Engineering)

Leadership Principles Demonstrated:

- **Ownership** - Stepped into leadership vacuum without being asked; owned team outcomes
- **Hire and Develop the Best** - Enabled Onkar's success through pairing and resource sharing
- **Insist on the Highest Standards** - Maintained quality through TOI/PRR processes
- **Think Big** - Managed both tactical execution and strategic stakeholder communication
- **Bias for Action** - Proactively redistributed work when constraints arose
- **Earn Trust** - Directors trusted me to represent team; Google team relied on me as single point of contact
- **Deliver Results** - Shipped AGA 2 on time while covering leadership for 1.5 months
- **Strive to be Earth's Best Employer** - Created supportive environment for teammates during transition

Situation

My team on the Google Cloud NetApp Volumes project was without a direct manager and reporting to our skip-level director. Unfortunately, tensions had escalated between one of my teammates and the director, leading to emotionally charged meetings that hindered productivity and communication. The team lacked leadership structure at a critical time when we needed to deliver on our commitments to Google.

Task

Recognizing the leadership vacuum and communication breakdown, I approached my director and volunteered to step into a team lead role. My goals were to: restore healthy team dynamics, establish clear communication channels with both the Google team and our director, organize our backlog into actionable work, and ensure we delivered on our commitments—particularly the MQoS (Managed Quality of Service) feature.

Action

I took on a hybrid 25/75 leadership/engineering role and implemented several key initiatives:

- Became the official liaison with the Google team, centralizing communication and reducing friction
- Organized the backlog by breaking down complex requirements into digestible stories mapped to appropriate epics
- Established daily scrums to track progress, identify blockers, and maintain team alignment
- Created a weekly reporting cadence with my director, providing consolidated status updates and keeping him informed without requiring direct team interaction
- Drove the MQoS and AGA 2 feature to completion, coordinating testing, contract negotiations with Google, and securing final approval
- Maintained my engineering contributions while balancing leadership responsibilities

Result

- Successfully delivered AGA 2 on schedule despite leadership gap and availability constraints
 - Successfully delivered the MQoS feature on time and to specification, meeting all contractual obligations with Google
 - Improved team dynamics significantly with structured communication and clear leadership
 - Provided my director with consistent, professional updates without the previous interpersonal tensions
 - Demonstrated capability for leadership roles by balancing technical delivery with people management in a challenging environment
 - Maintained seamless communication with Google team and internal stakeholders throughout the 1.5-month transition
 - Enabled teammate success through active collaboration and resource sharing with Onkar and others
 - Balanced 25% leadership with 75% engineering work, delivering on both fronts without dropping commitments
 - Demonstrated readiness for formal leadership roles as a Level 3 engineer performing at Level 4+ capacity
 - Proved ability to step up during organizational transitions and deliver results under pressure
-

Rapid Onboarding & Immediate Impact

Leadership Principles Demonstrated:

- **Customer Obsession** - Focused on understanding customer needs through cross-functional 1:1s
- **Ownership** - Took responsibility for own ramp-up; didn't wait to be spoon-fed
- **Learn and Be Curious** - Proactively scheduled 1:1s to learn product and organization
- **Insist on the Highest Standards** - Delivered production-quality 1,000-line PR in 6 weeks
- **Bias for Action** - Delivered comprehensive test plan within one week of starting
- **Earn Trust** - Built relationships across org; code still in use demonstrates quality
- **Dive Deep** - Gained architectural depth that enabled complex debugging later
- **Deliver Results** - Contributed to FlexCache Preview while ramping up
- **Strive to be Earth's Best Employer** - Invested in culture and welcoming environment

Situation

I rejoined NetApp in February after previous tenure with the company (5 years cumulative). Jake and I were new to the Google Cloud NetApp Volumes team and needed to ramp up quickly on a complex codebase to contribute to active FlexCache Preview development and CRR testing efforts. The team had aggressive timelines and couldn't afford slow onboarding.

Task

I needed to become productive immediately, gain deep familiarity with the CVS codebase and architecture, and begin delivering features within the first few months—all while building relationships across a team I was new to and establishing myself as a reliable contributor.

Action

- Ramped up extremely quickly, making significant contributions within 6 weeks
- Delivered a complex 1,000-line PR for the FlexCache internal workflow just 1.5 months in (still heavily used today)
- Completed a comprehensive test plan for CRR within one week of starting, demonstrating immediate understanding of testing requirements
- Continued delivering features throughout the FlexCache Preview epic through May
- Proactively built relationships through 1:1s with cross-functional partners (Karl, Mohammed, Hrishi, Oliver, Vignesh, Gaurav O, Praveen, and Abishek) to understand the product ecosystem
- Invested in team culture by volunteering at in-office events and fostering a positive, welcoming environment

Result

- Became a productive contributor in half the typical ramp time, delivering production code within 6 weeks
 - Created foundational infrastructure still in use today (FlexCache internal workflow)
 - Gained deep architectural knowledge that enabled me to take on complex features like SVM peering retry analysis and war room debugging
 - Established strong cross-functional relationships that facilitated collaboration throughout the year
 - Set the foundation for exceeding peer performance as a Level 3 engineer working at Level 4+ capacity
 - Demonstrated commitment to "thrive together" through relationship building and cultural contributions beyond core technical work
-

NetApp (2018-2023)

Sales Engineering/Engineering Meetup - Cross-Functional Collaboration Initiative

Leadership Principles Demonstrated:

- **Customer Obsession** - Created feedback loops to understand how customers use the product
- **Ownership** - Identified organizational gap and took initiative to solve it
- **Invent and Simplify** - Created simple biweekly meeting structure to bridge complex silos
- **Learn and Be Curious** - Sought to understand customer success stories and field experiences
- **Think Big** - Initiative led to creation of new NetApp product (Project ARgo)
- **Earn Trust** - Built bridges between previously siloed organizations
- **Dive Deep** - Learned customer use cases and selling points to inform engineering decisions
- **Success and Scale Bring Broad Responsibility** - Recognized need to connect engineering work to customer impact

Situation

Within NetApp, there existed significant organizational silos—engineering groups only interacted with engineering, sales only interacted with sales. Many engineers rarely heard about customer

success stories, making it difficult to see the value of the product being delivered. Feedback loops were completely missing or not being communicated down to engineering teams.

Task

Bridge communication gaps within our company to improve feedback loops between field teams and engineering, and inspire better design and product decisions within our organization by connecting engineering work to customer outcomes.

Action

- Created a biweekly sales engineering/engineering meetup where sales engineers would share field successes and engineering would share current development work
- Facilitated knowledge sharing sessions where engineering learned how customers were using the product and what the key selling points were
- Enabled sales engineers to learn about pipeline features and prepare to sell upcoming capabilities
- Maintained consistent meeting cadence and engagement across both organizations

Result

- Meetings ran successfully for over a year until organizational restructuring
- Led directly to the creation of Project ARgo, a new NetApp product leveraging AR to guide hardware installations
- Project ARgo won "Best Idea" at NetApp's innovation sprint competition
- When brought to the sales engineering/engineering meetup, sales engineers identified a need in a different organization
- Sales engineers connected engineering with leaders from that org, ultimately escalating to CEO George Kurian
- Secured CEO approval and budget to productize Project ARgo
- Created lasting impact by establishing a new product line at NetApp through cross-functional collaboration
- Demonstrated how breaking down silos can lead to innovation and business value

Filling in as Product Owner - 8-Month Leadership Transition

Leadership Principles Demonstrated:

- **Customer Obsession** - Redirected security approach based on customer feedback
- **Ownership** - Volunteered to fill critical role gap for 8 months
- **Learn and Be Curious** - Applied SAFe Product Owner training to real-world scenario

- **Think Big** - Managed product strategy while maintaining engineering contributions
- **Bias for Action** - Stepped up immediately when need was identified
- **Earn Trust** - Successfully delivered 2 PIs in hybrid role
- **Have Backbone; Disagree and Commit** - Challenged custom mTLS approach and advocated for Istio
- **Deliver Results** - Successfully delivered on product roadmap during transition period

Situation

While working as a Go backend engineer, my Product Owner was preparing for paternity leave and would be out of office for 5+ months. Our team was going to be without product leadership during a critical development period.

Task

The team needed an interim Product Owner to maintain product direction, stakeholder communication, and delivery momentum during the extended absence.

Action

- Volunteered to fill in as temporary Product Owner in a 50/50 capacity (engineer/product owner), ultimately serving ~8 months including soft handoff and handback periods
- Leveraged SAFe Product Owner training I had received in 2019 to apply product management skills
- Managed product backlog, stakeholder communications, and sprint planning while maintaining engineering contributions
- Successfully challenged the team's custom mTLS approach: scheduled meeting with Architects, PM, Engineering, and Sales Engineers to evaluate market landscape
- Advocated for adopting Istio instead of reinventing the wheel with a custom solution
- Presented data showing most of the market already used Istio, and custom solution was being designed for a small number of customer requests

Result

- Successfully delivered 2 Program Increments (PIs) as Product Owner
- Team adopted Istio-only recommendation after stakeholder alignment meeting
- Delivered more robust, generalist solution that aligned with market standards
- Prevented significant engineering investment in custom solution with limited applicability
- Demonstrated ability to balance product strategy with technical execution
- Proved capability to step into leadership roles and make strategic product decisions
- Smoothly transitioned role back to returning Product Owner with clear documentation and handoff

OpenShift POC - Leveraging Relationships for Accelerated Delivery

Leadership Principles Demonstrated:

- **Ownership** - Volunteered as lead engineer for critical infrastructure initiative
- **Invent and Simplify** - Leveraged IPI CLI to automate complex deployments
- **Are Right, A Lot** - Identified right tool (IPI) and right contact (Andrew Sullivan) to accelerate delivery
- **Learn and Be Curious** - Proactively reached out to network to learn about OpenShift solutions
- **Earn Trust** - Leveraged relationships built across NetApp and Red Hat
- **Dive Deep** - Understood deployment requirements and evaluated beta tooling
- **Deliver Results** - Accelerated OpenShift delivery tenfold through strategic networking

Situation

Our team was tasked with supporting multiple bare metal Kubernetes deployments, with highest priority being OpenShift integration. We needed to provide a proof of concept and integrate it into our CI/CD pipeline for automated testing.

Task

I volunteered as the lead engineer, working alongside a QA engineer and a DevOps engineer. We needed to provide a POC for installing OpenShift quickly and integrate it as a solution in our CI/CD pipeline to automate tests within these clusters.

Action

- Immediately reached out to a Solutions Architect at NetApp with whom I had developed a connection
- Recalled that he had mentioned colleagues moving to Red Hat, including Andrew Sullivan, a former Technical Marketing Engineer at NetApp
- Researched Andrew's work and found he had created numerous YouTube demos on the current OpenShift platform
- Arranged a call with Andrew, who provided valuable recommendations including a beta tool called IPI (Installer Provisioned Infrastructure)
- Evaluated IPI CLI for automation capabilities and deployment parameter configuration
- Integrated IPI into our CI/CD pipeline to automate OpenShift deployments with configurable parameters (nodes, masters/workers, nameserver, load balancer, etc.)

Result

- Successfully leveraged IPI CLI to automate deployments within CI/CD pipeline
- IPI provided exactly what we needed: CLI-based configuration of OpenShift deployment parameters with full automation
- Accelerated OpenShift delivery tenfold by leveraging network connections and identifying the right tool early
- Demonstrated value of building and maintaining cross-organizational relationships
- Enabled team to meet aggressive timeline for bare metal Kubernetes support
- Established repeatable, automated deployment process for ongoing testing and development

NgRx Revamp of UI - Rapid Full-Stack Transition

Leadership Principles Demonstrated:

- **Customer Obsession** - Improved UI to meet customer specifications and UX requirements
- **Ownership** - Took responsibility for own skill development to contribute to team needs
- **Learn and Be Curious** - Proactively sought mentorship and training to learn Angular
- **Insist on the Highest Standards** - Contributed to production code within one month
- **Bias for Action** - Immediately scheduled meetings and started training upon role change
- **Earn Trust** - Senior engineers invested time in mentoring; delivered on their confidence
- **Dive Deep** - Learned Angular, NgRx state management, and frontend best practices rapidly
- **Deliver Results** - Successfully contributed to NgRx integration effort within weeks

Situation

I had recently asked my manager to switch to full-stack development to gain more exposure to frontend development. The request was approved, and I started exploring our Angular frontend service. At this time, the team was revamping the UI to add support for NgRx state management.

Task

I needed to become proficient in Angular in a short period of time to help contribute to the frontend NgRx integration effort and not slow down the team's progress.

Action

- Scheduled meetings with Senior Frontend Engineers and any frontend newcomers within the organization to accelerate learning
- Arranged for Senior Engineers to provide deep dives on development workflows, frontend intricacies, and coding practices
- Completed Udemy courses on Angular using the company-provided subscription
- Applied learning immediately to real-world NgRx integration work
- Collaborated with senior engineers to understand state management patterns and best practices

Result

- Within less than one month, started contributing to the UI and helping with NgRx integration effort
- Successfully transitioned from backend-only to full-stack engineer
- Delivered production-quality frontend code while learning a new framework
- Found the experience of learning and applying a new skill in such a short period highly rewarding
- Established foundation for continued full-stack contributions
- Demonstrated ability to rapidly acquire new technical skills and apply them effectively

NgRx + Async/Await Issues - Complex Form Refactoring

Leadership Principles Demonstrated:

- **Customer Obsession** - Improved UX by resolving slow validations and poor form experience
- **Invent and Simplify** - Broke monolithic component into multiple focused components
- **Learn and Be Curious** - Ramped up quickly on frontend to solve complex state management issues
- **Dive Deep** - Understood component dependencies, validation flows, and NgRx patterns
- **Frugality** - Improved performance by distributing load across components
- **Deliver Results** - Successfully delivered revamped UI meeting customer specifications

Situation

While working as a full-stack engineer at NetApp, we were experiencing slow validations and poor UX on deployment forms. The original frontend form had many hidden aspects that would only become available after entries to initial questions (all on a single component). Some initial questions would trigger backend calls to validate deployment node compatibility, creating performance bottlenecks.

Task

Disseminate the monolithic form by breaking out dependent pieces into separate components. Feed information from previous components into dependent ones leveraging NgRx state management, with validations completing before navigating to the next form section.

Action

- Worked with senior engineers to create a template for NgRx components and understand how to feed information between components
- Leveraged NgRx and separate components to build out deployment information flow
- Implemented async validation pattern: wait on each component to execute backend validations, then navigate to next form with precompiled validations and options
- Ramped up quickly on frontend technologies to achieve this, despite limited prior frontend experience
- Applied NgRx state management patterns to maintain form state across multiple components

Result

- Successfully delivered a revamped UI that met customer specifications
- Removed dependency on a single monolithic component that led to poor UX
- Successfully broke out the form into multiple focused components with clear responsibilities
- Improved UX by providing validations and storing data with each page, lightening the load on individual components
- Leveraged peer knowledge, frontend templates, and NgRx documentation to achieve complex refactoring
- Demonstrated ability to tackle complex frontend architecture challenges despite being relatively new to frontend development

Onboarding Documentation - Organizational Impact Initiative

Leadership Principles Demonstrated:

- **Ownership** - Took responsibility for improving onboarding beyond own needs
- **Invent and Simplify** - Created simple, comprehensive documentation from complex tribal knowledge
- **Learn and Be Curious** - Documented learning journey to help future hires

- **Hire and Develop the Best** - Created resources that improved onboarding for 100+ new hires
- **Insist on the Highest Standards** - Maintained high documentation quality and completeness
- **Think Big** - Turned personal documentation into organization-wide initiative
- **Bias for Action** - Started documenting immediately rather than waiting for official process
- **Earn Trust** - Program Manager trusted documentation quality to build official onboarding program
- **Have Backbone; Disagree and Commit** - Challenged inadequate onboarding status quo
- **Deliver Results** - Created documentation used to onboard 100+ employees

Situation

While going through the onboarding process at NetApp, I found myself asking questions too often trying to figure out tribal knowledge that was not well documented. Many things specific to the role and auxiliary to it were not documented (for instance, how to print an ECO pass for public transit).

Task

Update the onboarding documentation to help make the onboarding process seamless for future hires and reduce the burden on existing team members answering repetitive questions.

Action

- Created verbose documentation detailing every aspect of my onboarding experience while going through the process
- Encouraged other new hires going through onboarding at the same time to contribute to the public documentation
- Created a Slack channel for all new hires to contribute new findings and help other new hires
- When the documentation caught the eye of our organization's Program Manager, collaborated with her to create an official initiative
- Provided my documentation as a jumping-off point to improve the onboarding experience
- Assisted in creating day-by-day itineraries for the onboarding journey
- Developed week-by-week checklists of learning items for new hires

Result

- Created comprehensive onboarding documentation that became the foundation for official onboarding program
- Documentation was used to onboard 100+ new hires

- Significantly reduced time spent by existing team members answering repetitive onboarding questions
 - Improved new hire experience and time-to-productivity
 - Established collaborative documentation culture among new hires through Slack channel
 - Demonstrated initiative and organizational impact beyond individual role
 - Program Manager adopted and expanded the work into official onboarding program
-

Greenfield GPU Support for VMware - VDI Infrastructure Development

Leadership Principles Demonstrated:

- **Customer Obsession** - Enabled customers to use cloud-based GPU workloads for architecture and design
- **Ownership** - Took full ownership of greenfield GPU support implementation
- **Invent and Simplify** - Integrated GPU capabilities into existing deployment infrastructure
- **Learn and Be Curious** - Researched SolidFire APIs, pyvmomi capabilities, and GPU specifications
- **Frugality** - Leveraged existing libraries and APIs rather than building from scratch
- **Dive Deep** - Understood GPU node specifications, VMware Horizon compatibility, and deployment requirements
- **Deliver Results** - Successfully delivered GPU-enabled node support with comprehensive testing

Situation

Product management informed us that we were developing a new SolidFire node that was GPU-enabled, codenamed Coruscant. The goal was to add support for VDI (Virtual Desktop Infrastructure) so that customers could use VMware deployments for cloud-based GPU-intensive workloads, such as architecture or graphic design work.

Task

Develop and support GPU-enabled nodes for VMware deployments, enabling customers to run GPU-intensive workloads in cloud-hosted environments.

Action

- Reached out to SolidFire team to validate that SolidFire APIs supported queries for GPU-enabled nodes, specifically determining which GPU was on the node and GPU specifications
- Explored deployment capabilities within pyvmomi (the infrastructure deployment library used in Python)
- Validated that pyvmomi had support for VDI deployments and integrated it into our service
- Added a backend GPU API to query nodes and validate GPU availability
- Implemented logic to handle two types of GPUs supported by Coruscent nodes, each limiting compatible VMware Horizon versions
- Created API to present available Horizon versions for installation to users based on GPU type
- Worked closely with QA engineers post-delivery to integrate extensive GPU tests with various graphics tests

Result

- Successfully delivered support for GPU-enabled nodes for VMware VDI deployments
- Enabled customers to run GPU-intensive workloads (architecture, graphic design) in cloud environments
- Created comprehensive API layer for GPU node queries and compatibility validation
- Integrated extensive testing with QA engineers to validate remote graphics capabilities
- Delivered greenfield feature that expanded product capabilities into new market segment
- Demonstrated ability to work across teams (SolidFire, QA) to deliver complex infrastructure features

MongoDB mTLS Support - Complex Integration Debugging

Leadership Principles Demonstrated:

- **Ownership** - Took responsibility for resolving difficult integration issue
- **Invent and Simplify** - Simplified complex mTLS integration through systematic debugging
- **Learn and Be Curious** - Researched Envoy proxy and MongoDB documentation extensively
- **Insist on the Highest Standards** - Refused to compromise on security integration quality
- **Bias for Action** - Engaged in extensive trial and error to find solution
- **Earn Trust** - Transparently communicated difficulties to team and sought help
- **Dive Deep** - Understood TCP/IP vs HTTP/HTTPS differences, Envoy configuration, and MongoDB TLS
- **Deliver Results** - Successfully resolved complex integration through collaboration and persistence

Situation

I was experiencing significant difficulty integrating our custom mTLS solution into MongoDB. We had created a template for integrating mTLS into services, but encountered unforeseen issues with MongoDB. Unlike most of our services using HTTP/HTTPS, MongoDB uses TCP/IP. MongoDB has its own TLS approach built into its service/yaml, and Envoy over TCP has various configuration parameters (tcp_keepalive, tcp type on filters) that differed from our template.

Task

Resolve issues related to integrating MongoDB with our custom mTLS solution, despite protocol differences and configuration complexities not covered by our existing template.

Action

- Leveraged `kubectl edit` extensively to create a trial-and-error loop of integrating Envoy into MongoDB service
- Referred extensively to documentation for both Envoy proxy and MongoDB to understand integration requirements
- Monitored traffic on Envoy and attempted connections via `mongosh --tlsCertKeyFile <> --tlsCAFile <>`, iterating on failures
- Raised the issue to the team during standup when experiencing continued difficulty
- Scheduled pair programming session with a colleague who had available cycles
- Brought colleague up to speed on documentation reviewed, attempts made, and desired outcome
- Collaborated in hour-long session, continuing trial and error with additional perspective
- Identified missing security aspects on the MongoDB yaml through collaborative debugging

Result

- Successfully resolved complex MongoDB mTLS integration after exhausting individual approaches
 - Demonstrated persistence through documentation review, trial and error, and seeking help
 - Proved value of pair programming for complex debugging scenarios
 - Created working integration despite protocol differences from standard template
 - Showed ability to recognize when to escalate and collaborate rather than continuing to struggle alone
 - Delivered secure MongoDB integration that met security requirements
-

CRD + Operator Greenfield Work - On-Premises Kubernetes Deployment

Leadership Principles Demonstrated:

- **Customer Obsession** - Enabled on-premises customers to deploy hybrid cloud management
- **Ownership** - Took ownership of complex greenfield operator development
- **Invent and Simplify** - Created operator-based solution to simplify complex multi-service deployments
- **Learn and Be Curious** - Researched and validated architect recommendations for on-prem solutions
- **Think Big** - Designed scalable deployment solution for private cloud infrastructure
- **Bias for Action** - Developed POC to validate approach before full implementation
- **Earn Trust** - Collaborated with architects and senior engineers on critical infrastructure
- **Dive Deep** - Understood Kubernetes operators, CRDs, and complex service orchestration
- **Deliver Results** - Successfully delivered operator-based on-premises deployment solution

Situation

Upon a reorganization at NetApp, I was moved to a hybrid cloud management application, specifically placed on the deployment team for on-premises clusters. My new team was tasked with replicating our public cloud service for private cloud deployments. Many aspects of public cloud deployments are provided by cloud providers (infrastructure management, secret management, HA, load balancing, object storage), but on-premises deployments required us to configure these ourselves. Architects determined commonly used solutions (Consul, Vault, MongoDB, Istio, InfluxDB) and designed specifications for on-premises deployments.

Task

Generate Kubernetes deployment solution for on-premises deployments. While we had existing public cloud deployments to reference, we needed more control for on-premises deployments with specific parameters. The solution required developing a Custom Resource Definition (CRD) and operator to control the installation flow.

Action

- Researched validity of architect and PM recommendations for on-premises infrastructure components

- Worked on proof of concept with peers to evaluate the use of an operator as a potential solution
- Designed a CRD for Astra deployments with appropriate specifications
- Leveraged bash scripts to deploy the initial service and initiate deployment of each service/operator in a phased approach
- Implemented operator logic to generate secrets and update service specifications so each service could leverage said keys
- Collaborated with senior engineers and architects through knowledge-sharing sessions to develop viable solution
- Addressed resource constraint challenges: Identified that Rancher deployment on Coruscent nodes lacked sufficient resources
- Implemented resource limits and leveraged Kubernetes taints to ensure pods were scheduled on nodes with available resources

Result

- Successfully developed operator-based solution for deploying on-premises Kubernetes instances
- Delivered working CRD and operator that managed complex multi-service deployments
- Enabled on-premises customers to deploy hybrid cloud management capabilities
- Leveraged POCs and group knowledge sharing to validate approach before full implementation
- Resolved resource scheduling challenges through proper resource limits and node taints
- Demonstrated ability to work on complex greenfield infrastructure projects
- Proved capability to collaborate with architects and senior engineers on critical platform development